HX-CCD22 Integration Report

Prepared for internal senior review.

Date: 2026-06-29

Device under test:

- Camera model: HX-CCD22 people counter
- Camera IP used in lab: `172.16.17.50`
- Local receiver/server IP used in lab: `172.16.16.118`
- Camera serial number observed: `221000002507152593`

Executive Summary

The HX-CCD22 can send passenger-counting data directly to our own server. We tested the camera against a local receiver and also through a public HTTPS ngrok URL. The camera successfully sent heartbeats, incremental count uploads, per-person event data, height data, stay-time data, and, after child mode was enabled, explicit adult/child aggregate fields.

The recommended production direction is:

- Use the camera HTTP push API for production ingestion.
- Use Increment mode for backend analytics.
- Store raw and normalized data in a backend database.
- Use PostgreSQL in production instead of SQLite.
- Treat adult/child analytics as height-based classification, not real age detection.

The separate `camera_total_receiver.py` dashboard mirrors the camera website by polling the camera web API. It is useful for local dashboard validation, but it is not the same as the production push ingestion path.

Receiver Files

File	Purpose	Storage	Main use
`camera_data_audit_receiver.py`	Receives push data from camera	SQLite	Own-server ingestion validation
`camera_total_receiver.py`	Polls camera web API and serves dashboard	Memory only	Local dashboard/camera website mirror

camera_data_audit_receiver.py

This is the main own-server receiver. It answers the question: can the camera send all observed push data to our server instead of Foorir?

It receives:

- `POST /api/camera/heartBeat`
- `POST /api/camera/dataUpload`

It exposes:

- `GET /health`
- `GET /api/config`
- `GET /api/status/current`
- `GET /api/dashboard`
- `GET /api/debug/report`
- `GET /api/debug/requests`
- `GET /api/debug/uploads`
- `GET /api/debug/events`
- `GET /api/debug/fields`
- `GET /api/debug/child-mode`

It stores data into SQLite:

- Raw JSON requests
- Heartbeats
- Count upload rows
- Device rows
- Per-person event rows

For lab testing, SQLite is enough. For production, use PostgreSQL.

camera_total_receiver.py

This is the dashboard/camera website mirror. It polls the camera internal web API:

```
http://172.16.17.50/api/passengerFlow/passengerFlowData/DisplayDataGet
```

This endpoint requires the `Sign` header:

```
Sign: 5121DE883D66396C02E02E2E17D44E12
```

This receiver:

- Polls the camera web API every second.
- Mirrors the camera website cumulative totals.
- Serves the fleet dashboard on port `8089`.
- Stores state only in memory.
- Does not persist totals to SQLite.
- Treats push packets as diagnostics only.

This file should not be treated as the production ingestion service. It is a local dashboard prototype and camera website mirror.

Data Paths

Path	Direction	Used by	Purpose
Camera push API	Camera -> our server	`camera_data_audit_receiver.py`	Production-style ingestion

Path	Direction	Used by	Purpose
Camera web polling API	Our server -> camera	`camera_total_receiver.py`	Mirror camera website totals
WebRTC URL	Browser/dashboard -> camera	dashboard stream modal	Local live stream attempt

Push API Data Observed

The push API sends JSON directly to our server.

Device and Network Fields

Field	Meaning
`sn`	Camera serial number
`hostName`	Camera host name
`macAddress`	Camera MAC address
`hwPlatform`	Hardware platform
`swRelease`	Firmware/software release
`connectionType`	Wired/wireless connection type
`ipAddress`	Camera-reported IP field
`ipAddressMethod`	DHCP/static mode
`timeZone`	Camera timezone offset

Count Upload Fields

Field	Meaning
`startTime`	Start of upload/statistics window
`endTime`	End of upload/statistics window
`time`	Camera event/report time
`in`	People entering in that upload/window
`out`	People exiting in that upload/window
`passby`	Pass-by count
`turnback`	Turnback count
`avgStayTime`	Average stay/dwell time
`len`	Number of per-person attribute rows

Adult and Child Fields

After child mode was enabled, the push payload included:

Field	Meaning
`inAdult`	Adult-height entries
`outAdult`	Adult-height exits
`inChild`	Child-height entries
`outChild`	Child-height exits
`passbyAdult`	Adult-height pass-by count
`passbyChild`	Child-height pass-by count
`turnbackAdult`	Adult-height turnback count
`turnbackChild`	Child-height turnback count

Per-person attributes

The tested firmware emitted `attributes[]`, not `eventList[]`.

Field	Meaning
`attributes[].eventType`	Event type
`attributes[].height`	Estimated height in cm
`attributes[].idIndex`	Event/person identifier
`attributes[].lineId`	Counting line id
`attributes[].personId`	Person identifier
`attributes[].stayTime`	Stay/dwell time in ms
`attributes[].timeStamp`	Event timestamp

Data Not Seen in Push Payloads

The tested push payloads did not include:

- Video stream
- Images
- GPS
- Bus id, route id, stop id
- Passenger identity
- Ticketing/payment data
- `eventList[]` from the API document

The camera performs counting on-device and sends metadata/count data. It does not send a video stream through the HTTP push API.

Camera Web API Data

The camera web API `DisplayDataGet` returned camera website style totals.

Observed fields:

Field	Meaning
`enter`	Camera website IN total
`leave`	Camera website OUT total
`pass`	Camera website pass-by total
`return`	Camera website turnback total
`stay`	Current stay/in-frame count
`stayTimePerPerson`	Average stay time
`age.*`	Age bucket summary
`gender.*`	Gender bucket summary
`workcard`	Workcard field from camera API

Example from Postman:


```
{
  "enter": 7,
  "leave": 5,
  "pass": 1,
  "return": 0,
  "stay": 0,
  "stayTimePerPerson": 1.6923075914382935,
  "result": {
    "code": "0",
    "isError": "false",
    "message": "Display data retrieved successfully"
  }
}
```


Public HTTPS / ngrok Test

We tested a public HTTPS path using ngrok:

```
https://passivism-dimness-gigabyte.ngrok-free.dev
```

Result:

- `/health` worked through ngrok.
- Camera heartbeat was received.
- Camera data upload was received.
- `attributes[]` person rows were received.
- `/api/status/current` showed the device online.
- The camera could send directly to our server through public HTTPS.

Conclusion: the camera does not need to send data to Foorir for our dashboard. If Address1 points to our backend, data can be received by our backend.

Increment vs Total Mode

Increment mode

Increment mode sends upload windows. Each row has `startTime` and `endTime`, and `in/out/passby/turnback` represent what happened in that window.

Example:

```
{
  "startTime": 1782470100,
  "endTime": 1782470160,
  "in": 2,
  "out": 0
}
```

This means 2 people entered during that window.

Total mode

Total mode was tested separately. The observed Total-mode uploads still looked like per-window/event counts rather than reliable cumulative totals.

Observed pattern:

```
id 1: in 3, out 1
id 2: in 0, out 1
id 3: in 1, out 1
id 4: in 1, out 0
```

If the payload were truly cumulative, we would expect values to keep increasing:

```
id 1: in 3, out 1
id 2: in 3, out 2
id 3: in 4, out 3
id 4: in 5, out 3
```

Recommendation: use Increment mode for production analytics.

Child Mode Test

Child mode was enabled and tested on 2026-06-29.

Child-mode report:

```
camera_explicit_adult_child_counts_seen: true
height_based_child_classification_possible: true
rows_with_explicit_adult_child: 127 / 127
in_adult: 107
out_adult: 99
in_child: 0
out_child: 0
```

Height summary:

```
child_height_threshold_cm: 140
person_event_rows: 107
rows_with_height: 107
min_height_cm: 143
max_height_cm: 184
avg_height_cm: 167.4
inferred_child_events: 0
inferred_adult_events: 107
```

Conclusion:

- Child mode works in the push payload.
- The camera emitted explicit adult/child fields.
- No child count appeared because all observed heights were 143 cm or above and the threshold was 140 cm.
- Adult/child classification is height-based, not true age detection.

- This should be explained clearly to the client.

Dashboard NO SIGNAL Explanation

The dashboard screenshot shows correct counts but `NO SIGNAL`. This is expected from the current prototype logic.

In `camera_total_receiver.py`:

- Counts are updated from camera web API polling.
- Card status is computed from heartbeat push freshness.

Therefore:

```
DisplayDataGet polling works -> counts update
Heartbeat push to port 8089 is missing/stale -> card says NO SIGNAL
```

This is not a counting failure. It is a status-source mismatch.

Recommended future dashboard fix:

- Show Count API status separately.
- Show Heartbeat Push status separately.
- Show Stream status separately.

Example:

```
Count API: Online
Heartbeat Push: Offline
Stream: Unknown/Unavailable
```

Production Recommendation

For production:

- Host a public HTTPS backend, for example `api.company.com`.
- Configure camera Address1 to our backend.
- Use `POST /api/camera/heartBeat`.
- Use `POST /api/camera/dataUpload`.
- Use Increment mode.
- Keep Supplement and Device Info enabled.
- Store raw JSON and normalized rows.
- Use PostgreSQL, not SQLite.
- Register each camera serial number to a bus and door position.
- For buses with two doors, aggregate two camera serial numbers into one bus total.

Dashboard Scope Recommendation

Recommended dashboard sections:

- Fleet overview
- Bus detail page
- Camera/device health page

- IN/OUT/passby/turnback totals
- Adult/child split from camera fields
- Height distribution from `attributes[].height`
- Stay-time analytics from `stayTime` and `avgStayTime`
- Raw payload/debug page for support

Risks and Limitations

- Adult/child means height-based classification.
- Short adults may be classified as children.
- Tall children may be classified as adults.
- Camera placement, height calibration, line placement, lighting, and occlusion affect accuracy.
- `IN - OUT` is only an occupancy estimate.
- Free ngrok URLs are not production infrastructure.
- The camera web API is useful for lab validation, but production should use push ingestion.

Appendix: Useful Test URLs

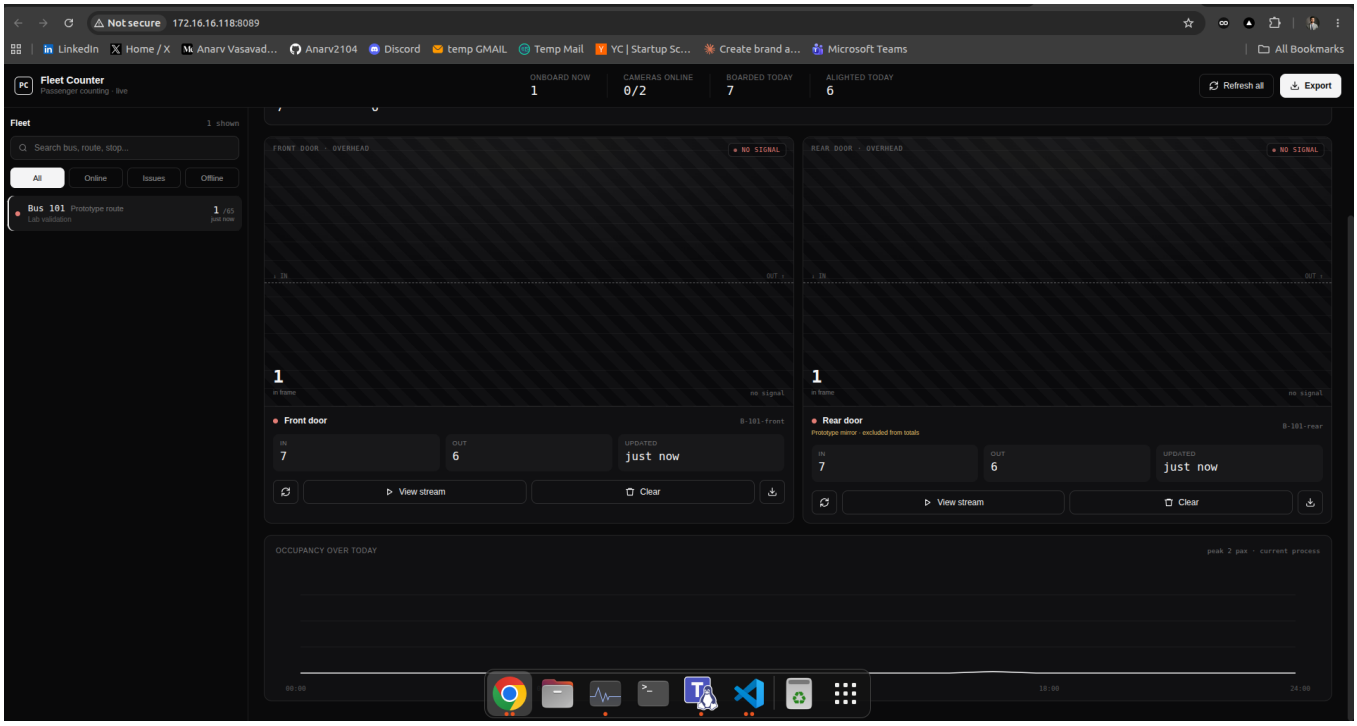
Audit receiver:

```
http://172.16.16.118:8088/health
http://172.16.16.118:8088/api/config
http://172.16.16.118:8088/api/status/current
http://172.16.16.118:8088/api/debug/report
http://172.16.16.118:8088/api/debug/requests
http://172.16.16.118:8088/api/debug/uploads
http://172.16.16.118:8088/api/debug/events
http://172.16.16.118:8088/api/debug/fields
http://172.16.16.118:8088/api/debug/child-mode
```

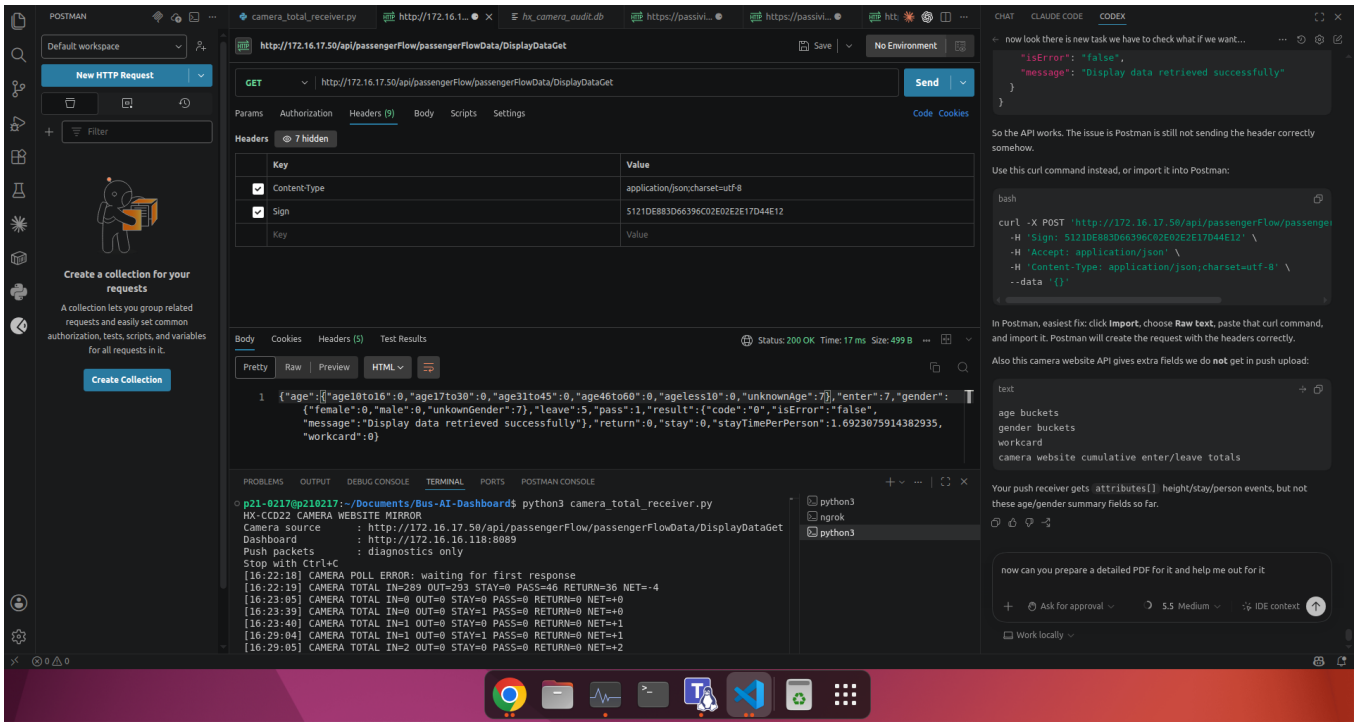
Camera web API:

```
POST http://172.16.17.50/api/passengerFlow/passengerFlowData/DisplayDataGet
Sign: 5121DE883D66396C02E02E2E17D44E12
Content-Type: application/json;charset=utf-8
Body: {}
```

Screenshots



Dashboard showing correct counts but NO SIGNAL



Postman DisplayDataGet success with Sign header